O'REILLY LIVE TRAINING · 2 HOURS

Claude Certified Developer – Foundations Exam Prep

Walk the CCDV-F blueprint, drill exam-style questions across all eight domains, and leave with a study plan.

Today's plan

SEGMENT	CLOCK	DOMAINS	WEIGHT
1 · The exam + blueprint	0:00–0:15	All	·
2 · API, applications, models	0:15–0:55	D2 + D5	49.9%
3 · Agents, tools, MCP	0:55–1:25	D1 + D8	25.3%
4 · Prompting, context, security, Claude Code, debugging	1:25–1:50	D6 + D7 + D3 + D4	24.8%
5 · Readiness map + study plan	1:50–2:00	All	·

Every segment ends with polls in the official item style.

SEGMENT 1 · 0:00 → 0:15 · 15 MIN

The exam and the blueprint

Logistics

8 domains · 25 weighted skills

How items are written

The exam at a glance

ITEM	DETAIL
Format	**53 items** · multiple choice + multiple response (item says how many)
Time	120 min · **~2 min 15 s per item** · plan ~135 min in the seat
Pass	Scaled **720** on 100–1,000 · 720 ≠ 72% raw
Delivery	**$125** · Pearson VUE, online (OnVUE) or test center · closed book
Prereqs	None · recommended 1–5 yrs SWE + 6+ months with Claude
Validity	12 months · free non-proctored renewal

Eight domains, 25 weighted skills

#	DOMAIN	WEIGHT	~ITEMS OF 53
D2	**Applications and Integration**	**33.1%**	~18
D5	Model Selection and Optimization	16.8%	~9
D1	Agents and Workflows	14.7%	~8
D6	Prompt and Context Engineering	11.0%	~6
D8	Tools and MCPs	10.6%	~6
D7	Security and Safety	8.1%	~4
D3	Claude Code	3.1%	~2
D4	Eval, Testing, and Debugging	2.6%	~1

Three things the weights hide

1. **~20% is plain software engineering.**
   Requirements, life cycle, REST/JSON/async, version control, SDKs, websockets. No Claude feature named.

2. **Claude Code leaks out of D3.**
   CLAUDE.md and settings.json sit in D2. Hooks sit in D1 and D7. Skills sit in D8.

3. **D5 is mostly fundamentals.**
   Tokens, sampling, thinking, effort, SDK plumbing. Pure "Opus vs Sonnet vs Haiku" is only 2.7%.

How an item is built

THE STEM

- 2–3 sentence scenario
- One **deciding constraint**, stated plainly: "cost is the primary concern", "hidden text", "reusable across several apps"
- "Which approach **best** fits?"

THE RIGHT ANSWER

- Names the **purpose-built mechanism**: Batches API, input isolation + guardrails, an MCP server
- Enforces in code, not in prose
- Smallest change that meets the constraint

Underline the constraint before you read the options.

Five distractor patterns

PATTERN	SOUNDS LIKE
Swap the model	"Switch to a larger / the smallest model"
Prompt as policy	"Add a line to the system prompt asking..."
Misses the constraint	Faster when the stem said cheaper; temperature for a security problem
Stuff the context	"Paste the data into every request"
Overclaims a feature	"Built-in tools can reach any internal API"

You have two weeks and can study deeply for only two domains. Which pair covers the most exam weight?

A. Claude Code + Eval, Testing, and Debugging

B. Agents and Workflows + Tools and MCPs

C. Applications and Integration + Model Selection and Optimization

D. Prompt and Context Engineering + Security and Safety

C · Applications and Integration + Model Selection ≈ 50%

OPTION	WEIGHT
A	3.1 + 2.6 = **5.7%**
B	14.7 + 10.6 = **25.3%**
C	33.1 + 16.8 = **49.9%**
D	11.0 + 8.1 = **19.1%**

Half the exam is API mechanics, app design, and model fundamentals.

SEGMENT 2 · 0:15 → 0:55 · 40 MIN

API mechanics, application design, and model choice

D2 Applications and Integration · **33.1%**

D5 Model Selection and Optimization · **16.8%**

Half the exam

Practice: Q1–Q4 + official sample 1

What the exam asks

- **2.3 Claude API Mechanics (6.8%)**: messages, tools, streaming, vision, thinking, caching, third-party vendors, batch vs realtime
- **2.4 Software Engineering Foundations (7.4%)**: REST, JSON, async, version control, SDLC, code review, refactoring
- **2.5 Claude Application Design (8.6%)**: instructions across interfaces, content boundaries, schema design, session hygiene, plugins
- **2.6 Configuration Management (4.1%)**: CLAUDE.md, settings.json, model version pinning, prompt versioning
- **5.1–5.4**: LLM fundamentals, SDK/REST/websockets, Opus vs Sonnet vs Haiku, token budgeting + caching

Skill names and weights from the CCDV-F Exam Guide v1.0, Section 6

One endpoint: `POST /v1/messages`

REQUEST

- `model`, `max_tokens`
- `system`: the behavioral contract
- `messages`: alternating user/assistant turns of **content blocks**
- `tools`, `tool_choice`
- `output_config`: `effort`, `format`
- `thinking`, `cache_control`

RESPONSE

- `content`: text, tool_use, thinking blocks
- `stop_reason`: why it stopped
- `usage`: input, output, `cache_creation_input_tokens`, `cache_read_input_tokens`

Stateless: you resend the whole conversation every call.

Read `stop_reason` before you read `content`

STOP_REASON	MEANING	YOUR CODE
`end_turn`	Finished naturally	Use the answer
`tool_use`	Wants a tool	Run it, return `tool_result`, loop
`max_tokens`	Hit your cap mid-output	Don't parse; raise cap or continue
`stop_sequence`	Hit a custom stop string	Expected cut
`pause_turn`	Long server-tool turn paused	Send back to continue
`refusal`	Safety classifier declined	Handle; check `stop_details`
`model_context_window_exceeded`	Window filled mid-generation	Trim / compact history

An extraction service asks Claude for JSON. About 3% of responses fail to parse. Every failed response has `stop_reason: "max_tokens"`. What is the best fix?

A. Add "Always return complete, valid JSON" to the system prompt.

B. Check `stop_reason` before parsing and raise `max_tokens` for this route so long extractions can finish.

C. Switch to a larger model that writes JSON more reliably.

D. Lower the temperature to make output more deterministic.

B · Check stop_reason, raise the cap

OPTION	PATTERN	WHY IT FAILS
A	Prompt as policy	The model was cut off; it can't finish what the cap stopped
C	Swap the model	Same cap, same truncation
D	Misses the constraint	Temperature doesn't change length (and newest models reject it)

The trace already told you the cause. Read `stop_reason`.

Four request shapes

SHAPE	USE WHEN	WATCH FOR
Synchronous	Short calls, backend jobs	Timeouts on long outputs
Streaming (SSE)	A user is watching; long or large outputs	Assemble per block; handle drops
Async client	Concurrency without blocking your app	Still realtime; still rate-limited
Message Batches	Bulk, offline, nobody waiting	Up to 24 h; **50% off**; results out of order

Is a user waiting? Then realtime. Is it bulk and can it wait? Then batch.

Streaming without corrupting state

- Events: `message_start` → `content_block_start` → `content_block_delta`… → `content_block_stop` → `message_delta` (**stop_reason, usage**) → `message_stop`
- Accumulate **per content-block index**; tool inputs arrive as partial JSON
- Act only on **complete** blocks; never execute a half-streamed tool call
- On a dropped stream: treat what you have as partial, then retry or resume; don't commit it
- SDK helpers: `get_final_message()` / `finalMessage()`

A developer must process 10,000 documents overnight to produce a non-urgent analytics report. Cost is the primary concern, and results are not needed until the following morning. Which approach best fits the requirement?

A. Send every request synchronously through the Messages API in parallel to finish as quickly as possible.

B. Use the Message Batches API, which processes large asynchronous workloads within a 24-hour window at reduced cost.

C. Lower max_tokens on synchronous calls to minimize cost.

D. Switch to the smallest available model regardless of output quality.

B · Message Batches API

OPTION	PATTERN	WHY IT FAILS
A	Misses the constraint	Optimizes speed; per-token cost unchanged
C	Misses the constraint	Truncates output; doesn't touch batch vs realtime
D	Swap the model	"Regardless of quality" breaks the report

Latency-tolerant + high volume + cost = Batches.

Prompt caching: pay once for the stable prefix

FACT	VALUE
Match	Exact **prefix**, in order `tools → system → messages`
Marker	`cache_control: {type: "ephemeral"}` · max **4** breakpoints
TTL	**5 min** default · `"ttl": "1h"` option · reads refresh the timer
Price	Write **1.25×** (5 min) / **2×** (1 h) · read **~0.1×** base input
Minimum	Model-dependent prefix length; shorter silently won't cache
Verify	`usage.cache_read_input_tokens` > 0 on repeat calls

Stable content first, volatile content after the last breakpoint.

A support assistant sends a 25,000-token system prompt and an identical tool list on every request, with a `cache_control` breakpoint on the last system block. `cache_read_input_tokens` is always 0. The system prompt begins with `Current time: {now}`. What should the developer do?

A. Switch to the 1-hour cache TTL so entries live longer.

B. Add more breakpoints throughout the system prompt.

C. Move the timestamp out of the system prompt to after the last breakpoint, such as into the user turn.

D. Move the workload to the Message Batches API.

C · Move the volatile timestamp after the breakpoint

OPTION	PATTERN	WHY IT FAILS
A	Misses the constraint	Longer TTL on a prefix that changes every call still never hits
B	Overclaims a feature	Every breakpoint sits after the changing first line
D	Misses the constraint	Interactive support can't wait 24 h

A byte change at position N invalidates everything after N.

Structured output: enforce, then still check

ENFORCE THE SHAPE

- `output_config.format` with a `json_schema`: the response **is** valid JSON
- `strict: true` on a tool: tool **inputs** match the schema
- Prefill is gone on current models (400)

STILL CHECK

- `stop_reason` `refusal` or `max_tokens` → no valid object
- Schema-valid ≠ correct: validate values and ranges
- Parse with `json.loads`, never string matching
- First call on a new schema is slower (grammar compile, cached 24 h)

Vision, PDFs, Files, third-party clouds

INPUTS

- `image` and `document` blocks, placed **before** the text question
- Files API: upload once, reference by `file_id`
- PDF: page images + text; citations optional

BEDROCK · VERTEX · FOUNDRY

- Same Messages API shape, **platform client** and platform auth (IAM, ADC)
- Different model IDs and pricing
- Newest betas may land on the Claude API first: check availability

Application design + configuration management

CONCERN	THE EXAM'S ANSWER
Instructions across surfaces	API: **you** own the system prompt. claude.ai / Desktop: product prompt + projects. Claude Code: CLAUDE.md + settings
Content boundaries	Instructions in `system`; data in tagged blocks; untrusted input never promoted to instructions
Session hygiene	Fresh context per task; don't carry stale history across unrelated jobs
Model version pinning	Pin an exact model ID in prod; run evals before you move
Prompt versioning	Prompts live in git next to their evals; change one, rerun the other
Plugins	Bundle commands, skills, agents, hooks, MCP; declare dependencies

Model choice and reasoning are separate levers

TIER	ROLE	DEFAULT MOVE
Fable	Most capable: hardest reasoning, long agentic work	Only when evals demand it
Opus	Demanding work above Sonnet	Step up on eval evidence
Sonnet	Balanced default for production	**Start here**
Haiku	Fast, cheap, high volume	Step down when evals say quality holds

- **Adaptive thinking** + `effort` (low → max) per request · `budget_tokens` is deprecated (400 on newest models)
- Thinking tokens bill as **output** · display is omitted by default; request a summary
- **Fast mode**: same Opus, faster output, premium price
- Sampling params (`temperature`, `top_p`) rejected on the newest models

A latency-sensitive classifier labels incoming tickets into four categories. It runs on an Opus-tier model at high effort. An eval on 500 labeled tickets shows a Haiku-tier model at low effort meets the same accuracy bar. What should the team do?

A. Keep the Opus model and enable fast mode to cut latency.

B. Move the route to the Haiku model at low effort, keeping the eval as a regression gate.

C. Keep the Opus model and turn thinking up for even higher accuracy.

D. Move the classifier to the Message Batches API to cut cost.

B · Smallest model that passes the eval

OPTION	PATTERN	WHY IT FAILS
A	Misses the constraint	Faster, but pays a premium for capability the task doesn't need
C	Misses the constraint	More thinking, more latency and cost, no accuracy gain needed
D	Misses the constraint	"Latency-sensitive" rules out a 24-hour window

Evals move you up or down a tier. Nothing else does.

Cost levers, cheapest first

1. **Cache the stable prefix.**
   Free win. Verify with cache_read_input_tokens.
2. **Batch what can wait.**
   50% off for offline work.
3. **Trim input.**
   Prune tool output, fewer examples, only the MCP servers you use.
4. **Tune effort, then model tier.**
   Per route, on eval evidence.
5. **Track usage per task.**
   Input, output, cache write, cache read. Cost per completed task, not per call.

An internal assistant's bill doubled. 80% of input tokens on every call are an identical 20,000-token block of instructions and tool definitions. A nightly job re-scores the day's 40,000 conversations synchronously. Which TWO changes reduce cost without lowering quality? **(Select two.)**

A. Add a cache breakpoint after the stable instructions and tool definitions.

B. Move the nightly re-scoring job to the Message Batches API.

C. Paste the full knowledge base into each request so the model needs fewer turns.

D. Switch every route to the most capable model to reduce retries.

E. Raise `max_tokens` on all calls.

A + B · Cache the prefix, batch the nightly job

OPTION	PATTERN	WHY IT FAILS
C	Stuff the context	Multiplies input tokens on every call
D	Swap the model	Higher per-token price everywhere, no evidence
E	Misses the constraint	A cap, not a cost lever

Multiple-response items: the stem tells you how many. Pick exactly that many.

SEGMENT 2 COMPLETE

Break · 5 min

SEGMENT 3 · 0:55 → 1:25 · 30 MIN

Agents, tools, and MCP

D1 Agents and Workflows · **14.7%**

D8 Tools and MCPs · **10.6%**

A quarter of the exam

Practice: Q5–Q8 + official sample 3

Workflow or agent?

WORKFLOW

- Your code owns the steps
- Prompt chaining, routing, parallelization, evaluator-optimizer
- Predictable cost and latency

AGENT

- The model picks the next step in a loop
- Only when **all four** hold: complex, valuable, viable for Claude, errors recoverable
- Supervisor + subagents for parallel or isolated work

If you can draw the flowchart in advance, it's a workflow.

The tool-use loop: Claude asks, your code runs

```
while True:
    resp = client.messages.create(model=MODEL, max_tokens=4096,
                                  tools=tools, messages=messages)
    messages.append({"role": "assistant", "content": resp.content})  # keep ALL blocks
    if resp.stop_reason != "tool_use":
        break
    results = [{"type": "tool_result", "tool_use_id": b.id,
                "content": run(b.name, b.input)}           # is_error=True on failure
               for b in resp.content if b.type == "tool_use"]
    messages.append({"role": "user", "content": results})  # ONE user turn, all results
```

- Every `tool_use` id answered by a matching `tool_result` in the **next** user turn
- Thinking blocks go back **unchanged**; never strip them mid-loop

An agent calls `get_order_status`. The downstream service times out. What should the developer's code send back to Claude?

A. Nothing; remove the `tool_use` block from history and retry the whole request.

B. A `tool_result` with an empty string so the loop continues.

C. A `tool_result` for that `tool_use_id` with `is_error: true` and a short message such as "Order service timed out; retry or ask the user."

D. A new user message telling Claude the tool is broken and to stop using tools.

C · tool_result with is_error: true

OPTION	PATTERN	WHY IT FAILS
A	Breaks the protocol	Editing history drops context and can invalidate thinking blocks
B	Masks failure	Claude reads "empty" as a real answer and may invent one
D	Prompt as policy	Unpaired tool_use fails validation; overreacts to one failure

Errors are information. Give the model a recoverable signal.

Tool definitions Claude selects correctly

PART	DO
`name`	Specific: `get_account_balance`, not `get_data`
`description`	3–4 sentences: what it does, **when to use, when not to**, what it returns
`input_schema`	`required` only for truly required fields, or Claude invents values
`strict: true`	Guarantees inputs match the schema
Overlap	Exclusion conditions on **both** tools, or merge into one tool with a type parameter
`tool_choice`	`auto` (default) · `none` · forced `any` / `tool` returns 400 on newest models
Parallel	Default on; `disable_parallel_tool_use` for one call per turn

Four ways to build an agent

APPROACH	YOU WRITE	WHO HOSTS
Manual loop (Messages API)	The whole loop + every tool	You
Tool Runner (SDK helper)	Just the tool functions	You
Claude Agent SDK	A prompt + options; get Claude Code's harness, built-in tools, hooks, subagents	You
Managed Agents	Agent config; Anthropic runs loop + sandbox	Anthropic

Frameworks (Strands, LangGraph, PydanticAI) wrap the same loop.

A team wants a coding agent that reads and edits files, runs shell commands, delegates to subagents, and blocks dangerous commands with hooks. Compliance requires it to run inside the team's own VPC. Which approach best fits?

A. The Claude Agent SDK, deployed on the team's infrastructure.

B. Managed Agents, so Anthropic hosts the loop and the sandbox.

C. A hand-written Messages API loop with file, shell, and subagent tools built from scratch.

D. The Message Batches API with one request per coding task.

A · Claude Agent SDK, self-hosted

OPTION	PATTERN	WHY IT FAILS
B	Misses the constraint	Anthropic-hosted sandbox, not their VPC
C	Over-engineering	Rebuilds what the SDK already ships
D	Misses the constraint	Batches has no interactive tool loop

Harness vs deployment: the SDK gives the harness, you keep the deployment.

Built-in tool, custom tool, Skill, or MCP?

CHOOSE	WHEN	CUE IN THE STEM
Built-in tool	Anthropic already ships it (web search, code execution, Read/Edit/Bash)	"Search the web", "run code"
Custom tool	One app needs one action on your system	"This app needs to call…"
Skill	Packaged know-how: instructions + scripts, loaded on demand	"Teach it our procedure / format"
MCP server	Reusable connection to a system, shared across apps and clients	"Reusable", "several apps", "maintained independently"

Also: **CLAUDE.md** = always-on project context · **Hook** = must-happen deterministic action · **Subagent** = isolated context

A team needs Claude to call an internal inventory service exposed as a REST API. They want the capability to be reusable across several Claude applications and maintained independently of any one app. Which approach best fits?

A. Hard-code the inventory logic into each application's system prompt.

B. Build an MCP server that exposes the inventory operations as tools so multiple Claude applications can connect to it.

C. Paste the current inventory data into the context window on every request.

D. Rely on a built-in tool, since built-in tools can reach any internal REST API.

B · An MCP server

OPTION	PATTERN	WHY IT FAILS
A	Prompt as policy	Neither reusable nor maintainable
C	Stuff the context	No live access; wastes context
D	Overclaims a feature	Built-in tools don't reach arbitrary internal APIs

MCP in one slide

PRIMITIVES

- **Tools**: actions the model calls
- **Resources**: data the app attaches
- **Prompts**: templates the user picks
- Host → client → server; same tool loop as before

TRANSPORTS

- **stdio**: local subprocess
- **Streamable HTTP**: remote; old SSE-only is deprecated
- API MCP connector: **remote servers only**
- Tool definitions cost context even unused

A developer built an MCP server that runs locally over stdio. It works in Claude Code. They now want a backend service to use it through the Claude API's MCP connector, and the connection fails. What is the cause and fix?

A. The beta header is missing; adding it lets the connector launch the stdio server.

B. The connector only reaches remote servers; deploy the server over Streamable HTTP, or run your own MCP client via the SDK.

C. MCP servers can't serve API apps; rewrite the server as a Skill.

D. The tool descriptions are too long; shorten them so the connector accepts the server.

B · Connector = remote HTTP servers only

OPTION	PATTERN	WHY IT FAILS
A	Overclaims a feature	The header enables the connector; it can't spawn local processes
C	Misses the constraint	Skills teach procedures; they don't expose a live system
D	Misses the constraint	Description length doesn't affect transport

Subagents, hooks, and humans

TOOL	WHAT IT DOES	USE FOR
Subagents	Fresh context per side task; returns a condensed result	Parallel or noisy work. Pass constraints explicitly: they don't inherit your conversation
Hooks	Deterministic code on lifecycle events; `PreToolUse` can **block** a call	Must-happen and must-never-happen rules
Human-in-the-loop	Approval checkpoint in your harness	**Irreversible** actions: payments, deletes, sends, deploys

Memory across sessions lives in storage you control, not in an ever-growing context.

A Claude Code agent must never run `git push --force` or delete files outside the repo, even if a prompt or a fetched web page tells it to. Which control is most reliable?

A. A bold rule in CLAUDE.md: "NEVER force-push or delete files outside the repo."

B. A `PreToolUse` hook that inspects Bash commands and blocks those patterns, plus permission deny rules.

C. A `PostToolUse` hook that logs every command for weekly review.

D. Switch to a more capable model that follows CLAUDE.md more closely.

B · PreToolUse hook + deny rules

OPTION	PATTERN	WHY IT FAILS
A	Prompt as policy	A request, not enforcement; injection can override it
C	Watch, don't block	Records the damage after it happens
D	Swap the model	Model choice doesn't enforce anything

Prompts ask. Hooks and permissions enforce.

SEGMENT 3 COMPLETE

Q&A

SEGMENT 4 · 1:25 → 1:50 · 25 MIN

Prompting, context, security, and the small domains

D6 Prompt + Context · **11.0%**

D7 Security · **8.1%**

D3 Claude Code · **3.1%**

D4 Debugging · **2.6%**

A quarter of the exam

Practice: Q9–Q13 + official sample 2

Diagnose, then add one structural fix

WHAT YOU SEE	WHAT THE PROMPT IS MISSING
Wrong shape: prose instead of a label or JSON	**Output constraint** (or structured outputs)
Scope or tone drifts deeper into the session	A more specific **system prompt**
Right task, invented structure	**Few-shot examples** in tags
Fine on tested inputs, breaks on a variant	A constraint or example **for that variant**

A prompt that grows longer every pass means you skipped the diagnosis.

A ticket router expects exactly `BILLING`, `TECHNICAL`, or `ESCALATION`. Claude returns "Billing", "billing", and sometimes "This looks like a billing issue." The developer has already added three paragraphs describing each category. What is the best next step?

A. Add two more paragraphs of category descriptions and edge cases.

B. Turn on extended thinking at high effort.

C. Constrain the output to exactly one label from the fixed set (or a JSON schema), with two tagged examples showing the format.

D. Switch to a larger model.

C · Output constraint + few-shot format examples

OPTION	PATTERN	WHY IT FAILS
A	More words, no structure	Longer prompts drift toward longer, looser outputs
B	Misses the constraint	Reasoning isn't the problem; the shape is
D	Swap the model	Same missing constraint, higher bill

Context engineering: the window is a budget

PROBLEM	LEVER
Old tool output piling up	**Prune / clear** stale tool results (context editing)
Long session nearing the limit	**Compaction**: summarize earlier turns; keep compaction blocks
Side task floods the main thread	**Subagent** with its own window, returns a summary
State must survive sessions	**Memory** in external storage, loaded on demand
Unrelated next task	**Clear history** and start fresh
Many MCP servers attached	Connect only what you use; defer tool loading

Never "save context" by stripping thinking blocks mid tool loop.

A research agent works well for 30 minutes, then starts repeating searches and ignoring a scope constraint from the first user turn. About 70% of its context is raw search results. Which change best addresses the root cause?

A. Move to a model with a larger context window.

B. Clear stale tool results and compact history, put the scope rule in the system prompt, and run searches in subagents that return condensed findings.

C. Raise `max_tokens` so the agent can write longer plans.

D. Restart the agent every 30 minutes with an empty context.

B · Prune, compact, isolate, and anchor the rule

OPTION	PATTERN	WHY IT FAILS
A	Swap the model	A bigger bucket fills with the same bloat
C	Misses the constraint	Output cap isn't the problem
D	Loses state	Throws away progress and the constraint with it

Security: treat every input as data, not orders

- **Prompt injection** arrives in documents, web pages, emails, tool results
- **Isolate** untrusted content in tagged blocks; never promote it to instructions
- **Least privilege**: remove tools the task doesn't need before you guard them
- **Layer guardrails**: input screening → output validation → tool permissions → hooks → human approval
- **Data**: minimize PII, redact before sending, control retention, block exfiltration paths

A polite rule in the prompt is not a control.

A Claude-powered agent summarizes web pages submitted by end users. One page contains hidden text instructing the model to ignore previous instructions and reveal its system prompt. Which mitigation is most effective?

A. Raise the model's temperature so its behavior is harder to predict.

B. Treat retrieved page content as untrusted input, keep it separate from trusted instructions, and use guardrails or hooks so injected instructions cannot trigger sensitive actions.

C. Add a line to the system prompt asking users not to include malicious instructions.

D. Switch to a larger model that follows instructions more reliably.

B · Isolate untrusted input + enforce guardrails

OPTION	PATTERN	WHY IT FAILS
A	Misses the constraint	Temperature is irrelevant to injection
C	Prompt as policy	A polite request isn't enforceable
D	Swap the model	A more obedient model can be **more** susceptible

Secrets, keys, and identity

- Keys live in a **secret manager or env vars**, never in prompts, CLAUDE.md, repos, or client-side code
- **Backend proxy** between browsers/mobile apps and the API
- **Separate keys per environment** and workspace; least-privilege scopes
- **Rotate** on exposure; monitor usage per key
- MCP servers and tools hold **their own** credentials server-side; the model never sees them
- Prefer **workload identity** over long-lived static keys where available

A startup's React web app calls the Claude API directly from the browser with an API key in the JavaScript bundle. Usage spikes overnight from unknown IPs. What should they do?

A. Obfuscate the bundle so the key is harder to find.

B. Rotate the key now, move Claude calls behind a backend that holds the key in a secret manager, and monitor per-key usage.

C. Add a system prompt line telling Claude to refuse requests from unknown users.

D. Lower `max_tokens` to limit the cost of each abusive request.

B · Rotate, proxy, store in a secret manager, monitor

OPTION	PATTERN	WHY IT FAILS
A	Security by obscurity	Anything shipped to the browser is public
C	Prompt as policy	The model can't authenticate callers
D	Misses the constraint	Caps damage per call; key is still stolen

Claude Code in one slide (D3 · 3.1%)

PIECE	WHERE	JOB
CLAUDE.md	Managed → `~/.claude/` → project root → subfolders	Always-on context; `/init` writes the first one
settings.json	Managed → user → `.claude/settings.json` (shared) → `.claude/settings.local.json` (personal)	Permissions, hooks, env, model
Rules	`.claude/rules/`	Scoped instructions by path
Skills	`.claude/skills/<name>/SKILL.md`	On-demand procedures
Commands	Built-in + custom `/slash` commands	Reusable prompts
Agents	`.claude/agents/`	Subagents with own tools + context
Modes	default · acceptEdits · plan · auto · dontAsk · bypassPermissions	How much it asks first
Headless	`claude -p "…" --output-format json`	CI and scripts

A team wants every developer's Claude Code to be blocked from reading `.env` files. Individual developers also want personal model preferences that stay out of git. Where should each setting go?

A. Both in the project CLAUDE.md.

B. The `.env` deny rule in `.claude/settings.json` (committed); personal preferences in `.claude/settings.local.json`.

C. The `.env` deny rule in each developer's `~/.claude/settings.json`; preferences in the project CLAUDE.md.

D. A line in the system prompt asking Claude not to open `.env` files.

B · Shared rule in project settings, personal prefs in local settings

OPTION	PATTERN	WHY IT FAILS
A	Prompt as policy	CLAUDE.md is context, not an enforced permission
C	Wrong layer	Relies on every person configuring it; prefs leak into git
D	Prompt as policy	Not enforceable; can be overridden

Right layer: enforce in settings, share in the repo, keep personal stuff local.

Integration layer or model output?

ERROR	MEANING	RECOVERY
400 `invalid_request_error`	Your request is malformed	Fix the code; don't retry as-is
401 / 403	Auth or permission	Fix the key or scope
413	Request too large	Trim input
429 `rate_limit_error`	Over your limit	Back off; honor `retry-after`
500 / 529 `overloaded_error`	Server side	Retry with exponential backoff

Read the trace first: request payload → `stop_reason` → tool_use/tool_result pairing → only then the prompt.

After a refactor of the history-building code, 15% of agent requests fail with `invalid_request_error: tool_result block references unknown tool_use_id`. The team has rewritten the tool descriptions twice. What should they do next?

A. Rewrite the tool descriptions a third time with clearer exclusion conditions.

B. Switch to a more capable model that tracks tool IDs better.

C. Inspect failing request traces and fix the code so each `tool_result` echoes the exact `tool_use_id` in the next user turn.

D. Wrap the call in a retry loop with exponential backoff.

C · It's an integration bug: read the trace, fix the code

OPTION	PATTERN	WHY IT FAILS
A	Wrong layer	A 400 validation error is not a prompt problem
B	Swap the model	The API rejects the request before the model runs
D	Wrong recovery	Retrying a malformed request fails identically

1. Temperature 0 guarantees identical outputs.

2. Claude executes your custom tools on Anthropic's servers.

3. MCP tool definitions consume context even when no tool is called.

4. Batch results come back in the order you submitted them.

5. Thinking blocks can be summarized to save context mid tool loop.

6. Structured outputs guarantee a parseable object even when `stop_reason` is `max_tokens`.

#	ANSWER	WHY
1	**False**	Sampling; never guaranteed. Newest models reject the param
2	**False**	Your code runs client tools; only server tools run at Anthropic
3	**True**	Definitions load up front; defer or allowlist
4	**False**	Key results by `custom_id`
5	**False**	Return them unchanged or the next request fails
6	**False**	Truncation still breaks the object; check `stop_reason`

SEGMENT 5 · 1:50 → 2:00 · 10 MIN

Your readiness map and study plan

Self-assessment | Two-week plan | Registration

Readiness map: rate yourself 0–2

DOMAIN	WEIGHT	YOU'RE READY WHEN YOU CAN…
D2 Applications	33.1%	Pick sync/stream/async/batch; place cache breakpoints; read any `stop_reason`
D5 Models	16.8%	Justify a tier + effort with an eval; explain tokens, sampling, thinking
D1 Agents	14.7%	Say workflow vs agent; choose manual loop / SDK / Managed Agents
D6 Prompt + context	11.0%	Map a failure to its missing piece; name four context levers
D8 Tools + MCP	10.6%	Write a tool description; choose built-in / custom / Skill / MCP
D7 Security	8.1%	Isolate untrusted input; layer guardrails; keep keys server-side
D3 Claude Code	3.1%	Put a setting at the right layer; run headless
D4 Debugging	2.6%	Tell a 400 from a 429 from a model mistake

Two-week study plan

DAYS	FOCUS	MATERIAL
1–2	Blueprint + fundamentals	Exam guide · prep lesson 1 · Q1–Q4
3–5	API mechanics, caching, batches, streaming	Prep lesson 2 · docs · exercises 01–02
6–7	Models, effort, cost	Model docs · pricing · Q3–Q4 again
8–9	Agents, tools, MCP	Prep lessons 2–3 · exercises 03–04
10–11	Prompting, context, security	Prep lesson 4 · Q9–Q13
12	Claude Code, debugging	Prep lesson 3 · settings docs
13	Full timed practice	All repo questions, 2 min 15 s each
14	Review misses only	cheatsheet.md · community question banks

Build one small app end to end: API + one tool + caching + an eval + basic guardrails. The guide asks for it.

Official prep path → blueprint

PREP LESSON	TIME	DOMAINS IT FEEDS
1 · MSO Foundations	57 min	D5 fundamentals
2 · Prompting, Agents & Tool Use	209 min	D2, D6, D1, D8
3 · Claude Code, MCP & Integration	142 min	D3, D8, D2 config
4 · Production Engineering, Evals & Security	211 min	D4, D7, D5 cost
5 · Accelerators & IP Contribution	155 min	D2 life cycle, D8 packaging

anthropic-partners.skilljar.com/path/claude-certified-developer-foundations

Registering and exam day

1. **Register with a Partner Network email.**
   Partner Academy certification page → purchase → Pearson VUE scheduling.

2. **Online or test center.**
   Government ID, clean desk, webcam for OnVUE. Closed book.

3. **Reschedule 48 h ahead to be safe.**
   Retakes after 14, 30, then 90 days.

4. **~2 min 15 s per item.**
   Read the last line first, underline the constraint, strike the distractor patterns, flag and move on.

If you remember seven things

1. **Read stop_reason before content.**
   Truncation, refusal, tool_use, window full.
2. **User waiting? Realtime. Bulk that can wait? Batch.**
   50% off, up to 24 h.
3. **Stable prefix first, then cache.**
   Volatile bytes after the last breakpoint.
4. **Evals move you between tiers.**
   Start Sonnet; tune effort per route.
5. **Claude asks, your code runs.**
   Pair every tool_use; return errors with is_error.
6. **Reusable connection → MCP. Procedure → Skill.**
   Must-happen rule → hook.
7. **Prompts ask; code enforces.**
   Isolate untrusted input, least privilege, keys server-side.

RESOURCES · Q&A

Thank you. Questions?

- Official CCDV-F Exam Guide v1.0 (PDF)
- Official prep path · 5 lessons, ~13 h
- Claude API docs · Claude Code docs · MCP docs
- Course repo · questions, cheat sheet, exercises

Lucas Soares
O'Reilly Live Training